Multimodal text image application

Multimodal text image application
1. Concept introduction
1.1 **What is text-to-image (Text-to-Image)?**
The core principle
1.2 **What is FastSDCPU?**
The core characteristics
The applicable scenarios
2. Project deployment
2.1 The deployment environment
2.2 LAN access
2.3 Using the text-to-image function
2.4 How to start it after a successful deployment
3. The online large model text-to-image (text/voice interaction)
3.1 Configuring Tongyi Wanxiang
3.2 The online text input
3.3 The online voice input

Because ollama does not support the text-to-image function, we need another tool to implement the local text-to-image function.

---

1. Concept introduction

1.1 What is text-to-image (Text-to-Image)?

Text-to-image is an artificial intelligence technology that automatically generates the corresponding image from a **text description**. You only have to enter a piece of text (for example "a Shiba Inu wearing sunglasses surfing on the beach"), and the AI model generates a picture that matches the description based on its semantic understanding; no drawing or design background is needed.

The core principle

1. **Language understanding**: the AI first parses your text and identifies the keywords (such as "Shiba Inu", "sunglasses" and "beach").
2. **Image generation**: trained on massive image data, the AI turns the text into visual elements and combines them into a reasonable picture.
3. **Style adaptation**: a style can be specified (realistic, cartoon, watercolour and so on), and the AI adjusts the generated result accordingly.

1.2 What is FastSDCPU?

FastSDCPU is an open source project for **Stable Diffusion text-to-image** that is optimised specifically for **CPU devices**. Through algorithmic and engineering optimisations it generates high quality images quickly on an ordinary computer without a GPU, which significantly lowers the hardware threshold for AI drawing.

The core characteristics

1. **Runs on the CPU alone**: no discrete graphics card is needed, and it runs directly on a laptop, a desktop or an edge device.
2. **Fast inference**: combined with technologies such as Latent Consistency Models (LCM), the image is generated in only 4–8 steps, which greatly shortens the waiting time.
3. **Lightweight deployment**: it supports the quantisation of the model (such as INT8) and the OpenVINO acceleration, which reduces the resource usage and improves the response efficiency.

The applicable scenarios

- A local AI creation tool
- An image generation service on an enterprise intranet
- Teaching demonstrations and prototype development
- A lightweight deployment in a resource constrained environment

2. Project deployment

2.1 The deployment environment

Note: if you are using our factory image, you do not have to deploy the environment; skip the deployment steps and go directly to <2.4 How to start it after a successful deployment> at the bottom and start it directly.

Open a terminal and run the following code:

```
# If git is not on the board yet, run this first
sudo apt update
sudo apt install git -y
sudo apt install python3.10-venv -y

# Add the environment variable
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

# Clone the project code
git clone https://github.com/rupeshs/fastsdcpu.git
cd fastsdcpu

# Create the virtual environment and install the dependencies
python -m venv venv
source venv/bin/activate

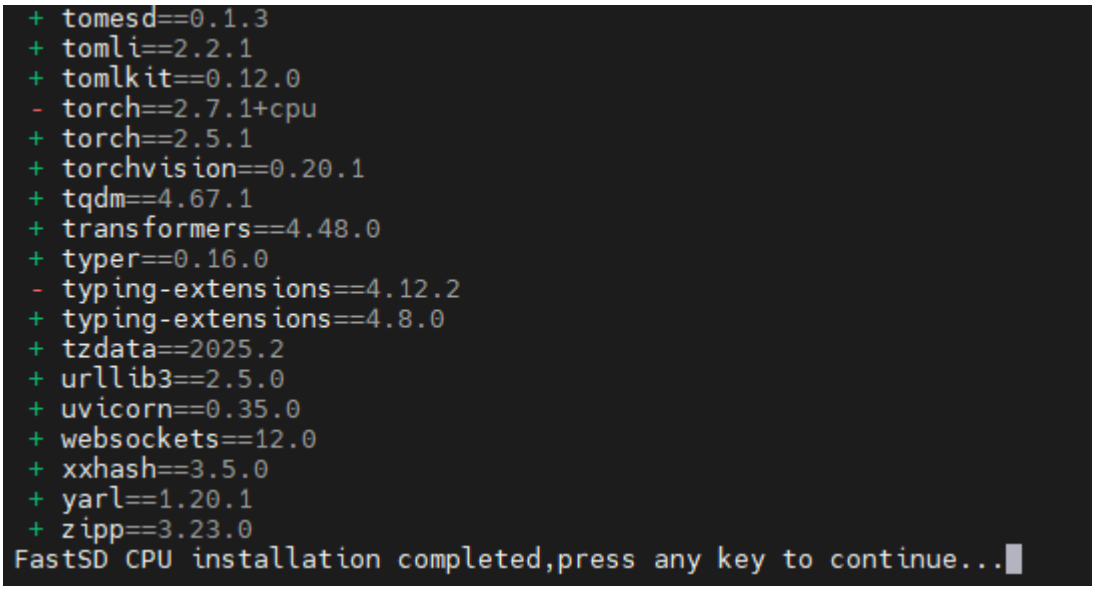
# Install uv
curl -Ls https://astral.sh/uv/install.sh | sh (this step may fail in China if no proxy is
used; if it does not succeed you can skip this step and run the next command)

# If the previous step did not install uv successfully with the curl command, run these
three commands
```

```
wget https://mirrors.huaweicloud.com/astral/uv/0.8.4/uv-aarch64-unknown-linux-gnu -O
~/.local/bin/uv
chmod +x ~/.local/bin/uv
~/.local/bin/uv --version

chmod +x install.sh start-webui.sh
./install.sh --disable-gui
```

The installation succeeded; press any key to exit:



2.2 LAN access

Before starting, one file has to be modified so that the LAN access is supported, otherwise the webui can only be accessed locally:

```
vim ~/fastsdcpu/src/frontend/webui/ui.py
```

After opening the ui.py file, go to the last line, find the line of code **webui.launch(share=share)** and change it to **webui.launch(server_name="0.0.0.0",share=share)**

Then save it

Start it:

```
./start-webui.sh
```

```
Found 7 stable diffusion models in config/stable-diffusion-models.txt
Found 4 LCM-LoRA models in config/lcm-lora-models.txt
Found 10 OpenVINO LCM models in config/openvino-lcm-models.txt
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
 UserWarning: Overwriting tiny_vit_5m_224 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny_
vit_5m_224. This is because the name being registered conflicts with an existing name. Please check if this is not exp
ected.
  return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
 UserWarning: Overwriting tiny_vit_11m_224 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny
_vit_11m_224. This is because the name being registered conflicts with an existing name. Please check if this is not e
xpected.
  return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
 UserWarning: Overwriting tiny_vit_21m_224 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny
_vit_21m_224. This is because the name being registered conflicts with an existing name. Please check if this is not e
xpected.
  return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
 UserWarning: Overwriting tiny_vit_21m_384 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny
_vit_21m_384. This is because the name being registered conflicts with an existing name. Please check if this is not e
xpected.
  return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
 UserWarning: Overwriting tiny_vit_21m_512 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny
_vit_21m_512. This is because the name being registered conflicts with an existing name. Please check if this is not e
xpected.
  return register_model(fn_wrapper)
Starting web UI mode
No lora models found, please add lora models to /home/sunrise/fastsdcpu/lora_models directory
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/gradio/components/dropdown.py:226: UserWarning: The value pas
sed into gr.Dropdown() is not in the list of choices. Please update the list of choices to include:  or set allow_cust
om_value=True.
  warnings.warn(
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/gradio/components/base.py:194: UserWarning: show_label has no
 effect when container is False.
  warnings.warn("show_label has no effect when container is False.")
* Running on local URL:  http://0.0.0.0:7860

To create a public link, set `share=True` in `launch()`.
```

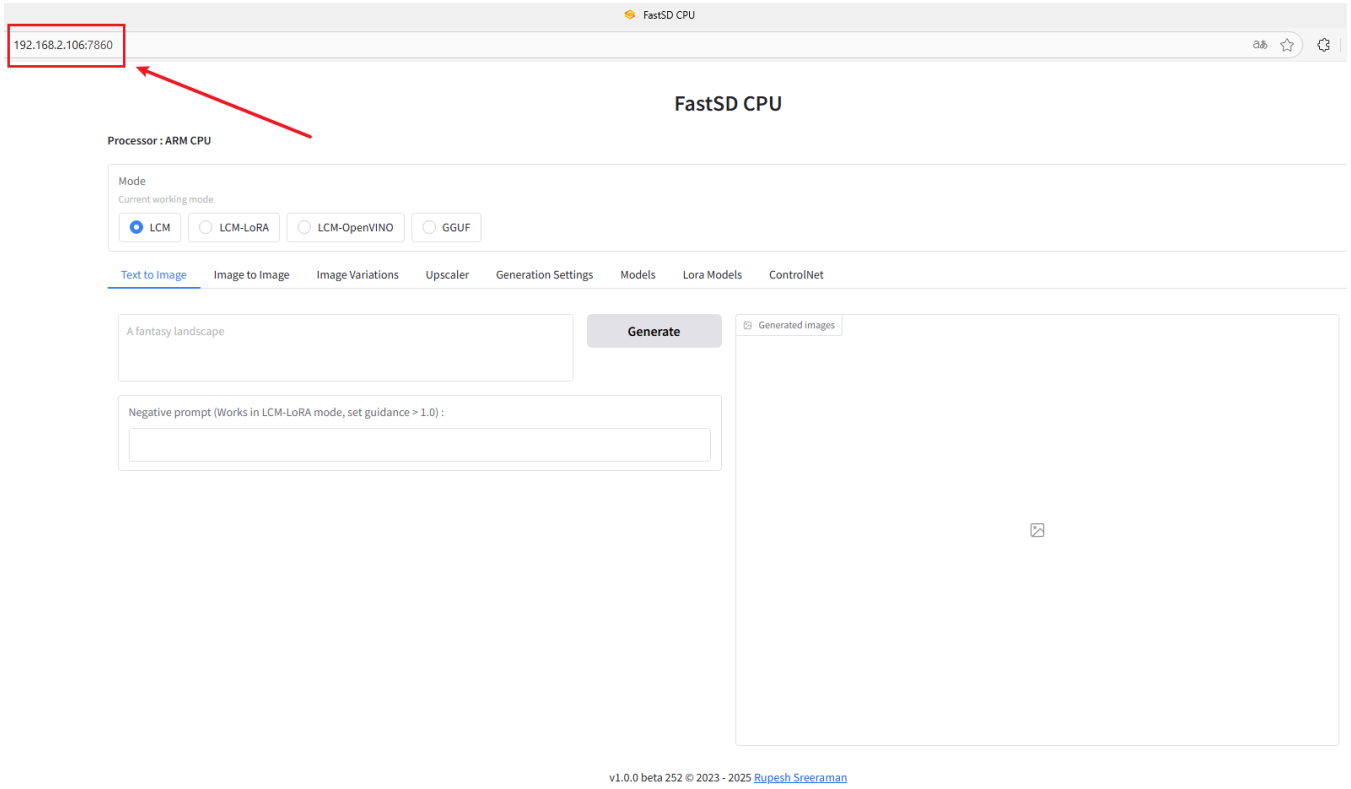
Then you can enter your mainboard IP:7860 in the browser to access this webui.

> The local text-to-image requires the Jetson mainboard to run the model, so the generation speed depends on the computing power of the mainboard and is usually slower than an online text-to-image service; the size of the model and the number of generation steps also affect the waiting time.

2.3 Using the text-to-image function

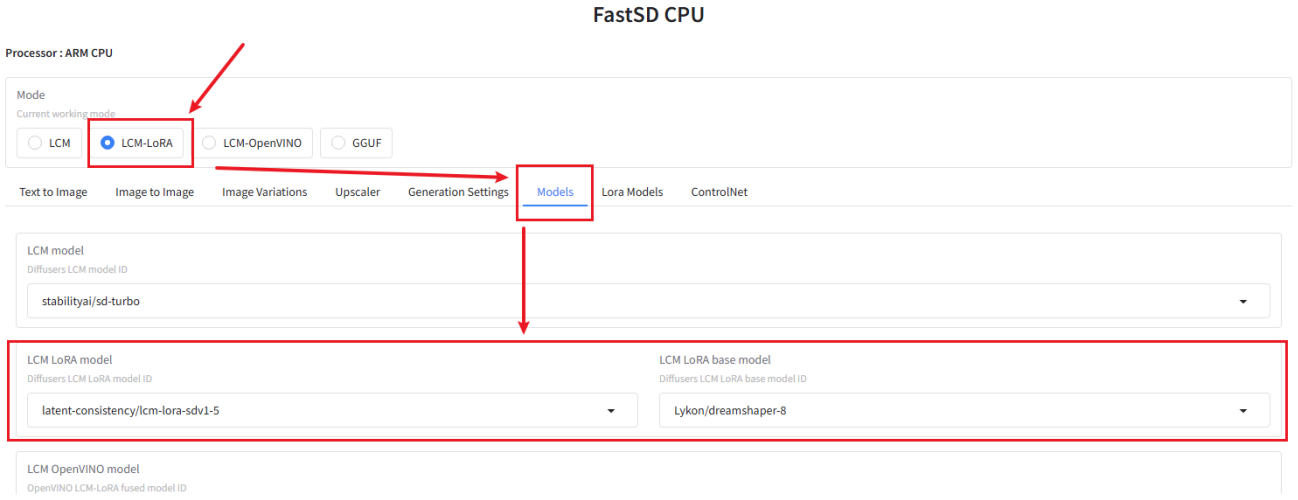
Use ifconfig in the terminal to query the IP of our mainboard, for example mine is 192.168.2.106.

Then open the browser and enter **your mainboard ip:7860**. For example, mine would be 192.168.2.106:7860, and that takes you into the webui.

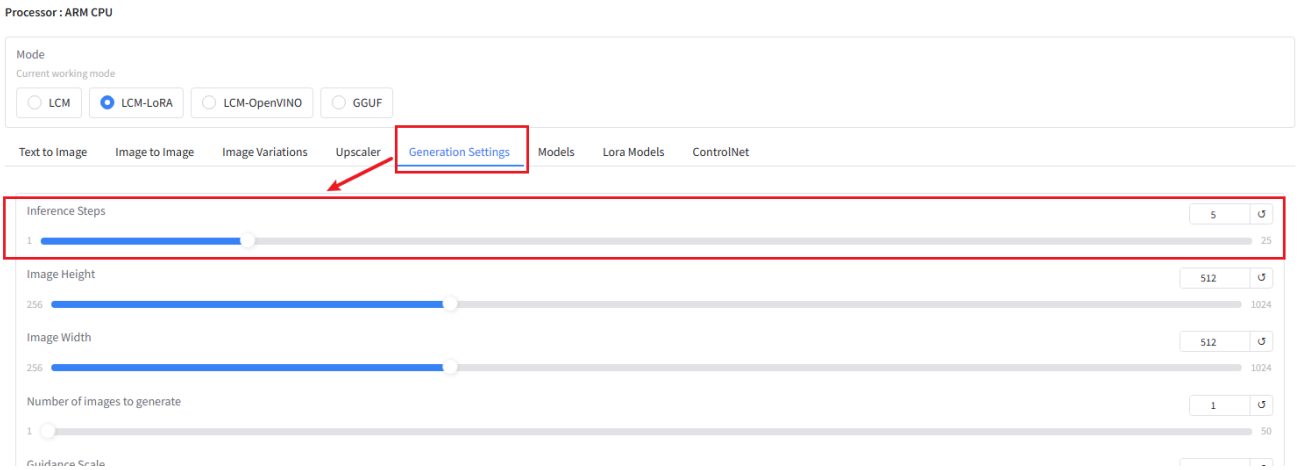


Then click LCM-LoRA; this model uses relatively little memory. If you want to use another model, you can look into it yourself.

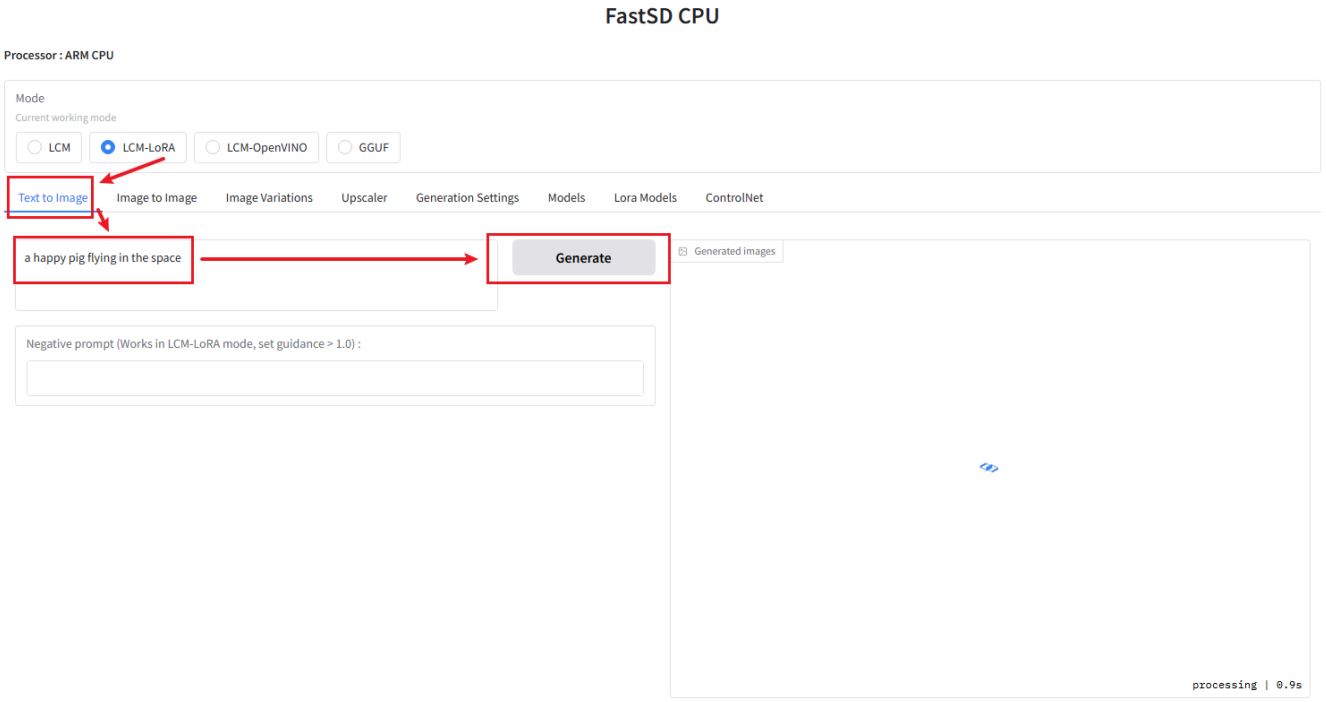
Then click Models to see the model settings of the LCM LoRA; you can change it to the model you want, or you can just choose the default as I did.



Then click Generation Settings and raise the Inference Steps, which improves the quality of the generated image; I have set it to 5 here.



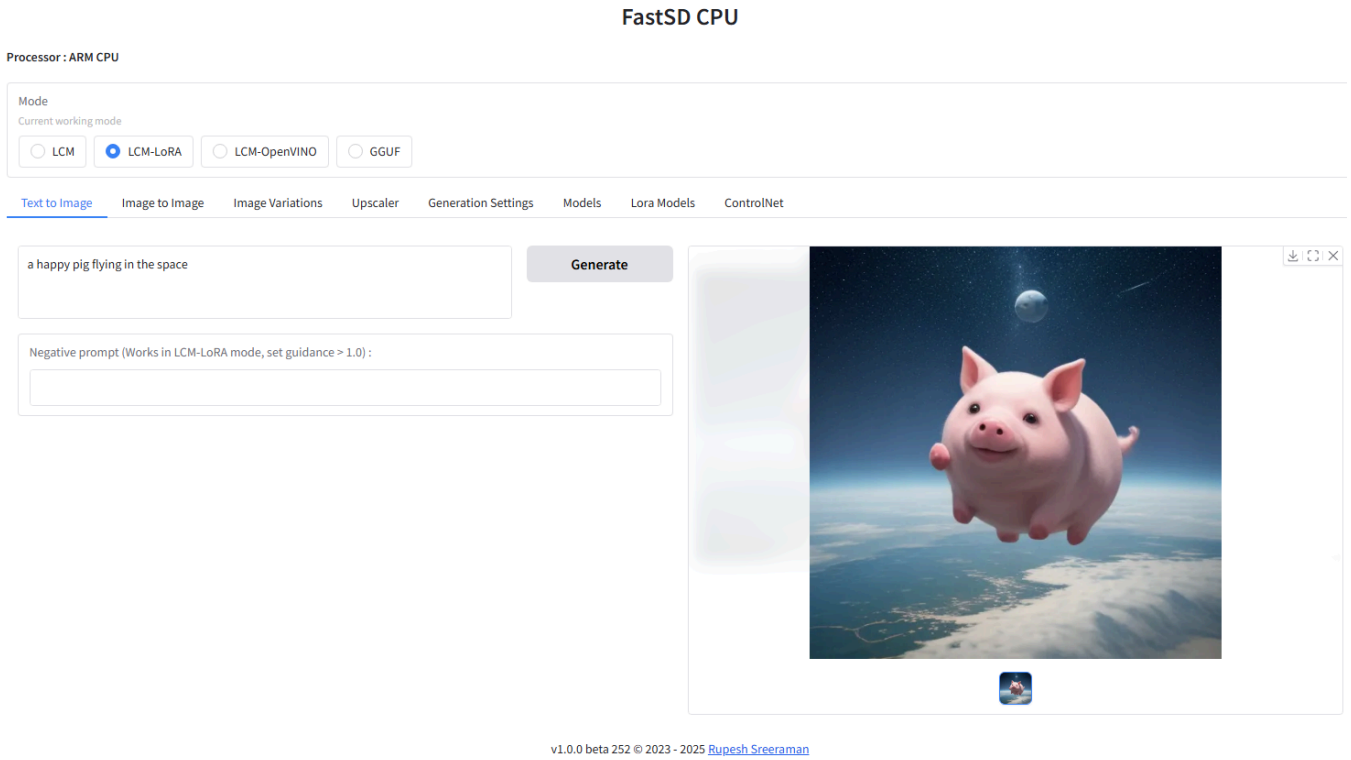
Then go back to our Text to Image, enter the content you want to generate in the dialog box, and press Generate to start generating the image.



On the first use the model has to be downloaded; in the terminal you can see the model that was selected by default being downloaded, and once the download has finished the text-to-image function starts running.

```
 'init_image': None,
 'lcm_lora': {'base_model_id': 'Lykon/dreamshaper-8',
              'lcm_lora_id': 'latent-consistency/lcm-lora-sdv1-5'},
 'lcm_model_id': 'stabilityai/sd-turbo',
 'lora': {'enabled': False,
          'fuse': True,
          'models_dir': '/home/sunrise/fastsdcpu/lora_models',
          'path': '',
          'weight': 0.5},
 'negative_prompt': '',
 'number_of_images': 1,
 'openvino_lcm_model_id': 'rupeshs/sd-turbo-openvino',
 'prompt': 'a happy pig flying in the space',
 'rebuild_pipeline': False,
 'seed': 123123,
 'strength': 0.6,
 'token_merging': 0.0,
 'use_gguf_model': False,
 'use_lcm_lora': True,
 'use_offline_model': False,
 'use_openvino': False,
 'use_safety_checker': False,
 'use_seed': False,
 'use_tiny_auto_encoder': False}
***** Init LCM-LoRA pipeline - Lykon/dreamshaper-8 *****
model_index.json: 100%|| 642/642 [00:00<00:00, 2.80MB/s]
config.json: 100%|| 796/796 [00:00<00:00, 3.81MB/s]
config.json: 100%|| 724/724 [00:00<00:00, 3.74MB/s]
special_tokens_map.json: 100%|| 472/472 [00:00<00:00, 2.27MB/s]
merges.txt: 525kB [00:00, 6.20MB/s]                 | 0.00/472 [00:00<?, ?B/s]
preprocessor_config.json: 100%|| 520/520 [00:00<00:00, 2.53MB/s]
tokenizer_config.json: 100%|| 737/737 [00:00<00:00, 3.54MB/s]
config.json: 1.87kB [00:00, 5.15MB/s]               | 0.00/1.22G [00:00<?, ?B/s]
vocab.json: 1.06MB [00:00, 9.26MB/s]                | 0.00/520 [00:00<?, ?B/s]
scheduler_config.json: 100%|| 614/614 [00:00<00:00, 2.99MB/s]
config.json: 100%|| 756/756 [00:00<00:00, 3.99MB/s]
model.safetensors:   2%|| 10.5M/492M [00:00<00:23, 20.3MB/s]
model.safetensors:  32%|| 157M/492M [00:36<01:24, 3.98MB/s]
model.safetensors:  12%|| 147M/1.22G [00:34<04:28, 3.98MB/s]
diffusion_pytorch_model.safetensors:  41%|| 136M/335M [00:34<00:50, 3.96MB/s]
diffusion_pytorch_model.safetensors:   4%|| 136M/3.44G [00:34<13:52, 3.97MB/s]
```

The generated result:



In practice this project supports English better, and the generated image matches the text more closely. It is recommended to use an English description to generate the image.

2.4 How to start it after a successful deployment

```
cd fastsdcpu # Enter the fastsdcpu directory
source env/bin/activate # Enter the virtual environment
./start-webui.sh # Start the webui
```

Once the webui has started successfully, enter your mainboard IP:7860 in the browser and you can start using the text-to-image function.

---

3. The online large model text-to-image (text/voice interaction)

The local FastSDCPU path is operated through a web page and does not support voice control; the online text-to-image calls Tongyi Wanxiang through `largemodel` and can use a text or a voice input. The online and the local are two independent ways of running; choose the one you need.

3.1 Configuring Tongyi Wanxiang

First apply for the Tongyi Qianwen API Key as described in the online large model basic tutorial and confirm that the account has available quota. Then open the model interface configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

Find the Tongyi Qianwen configuration item and fill in the API Key and the online text-to-image model. The text-to-image uses the `tongyi_media_model` parameter, not the `qianwen_model` that is used for the visual question answering:

```
qianwen_api_key: "Replace it with your own API-Key"
tongyi_media_model: "flux-schnell"
```

After saving it, open the main configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

Select the Tongyi platform in the existing `model_service.ros__parameters` parameter block; if `llm_platform` already exists, modify its value and do not add it again:

```
model_service:
  ros__parameters:
    llm_platform: 'tongyi'
```

After saving both configuration files, restart `largemodel` so that the settings take effect. The online call requires the Jetson to be able to reach the Tongyi service, a valid API Key and an account with available quota. Never publish a real API Key or show it in a screenshot.

3.2 The online text input

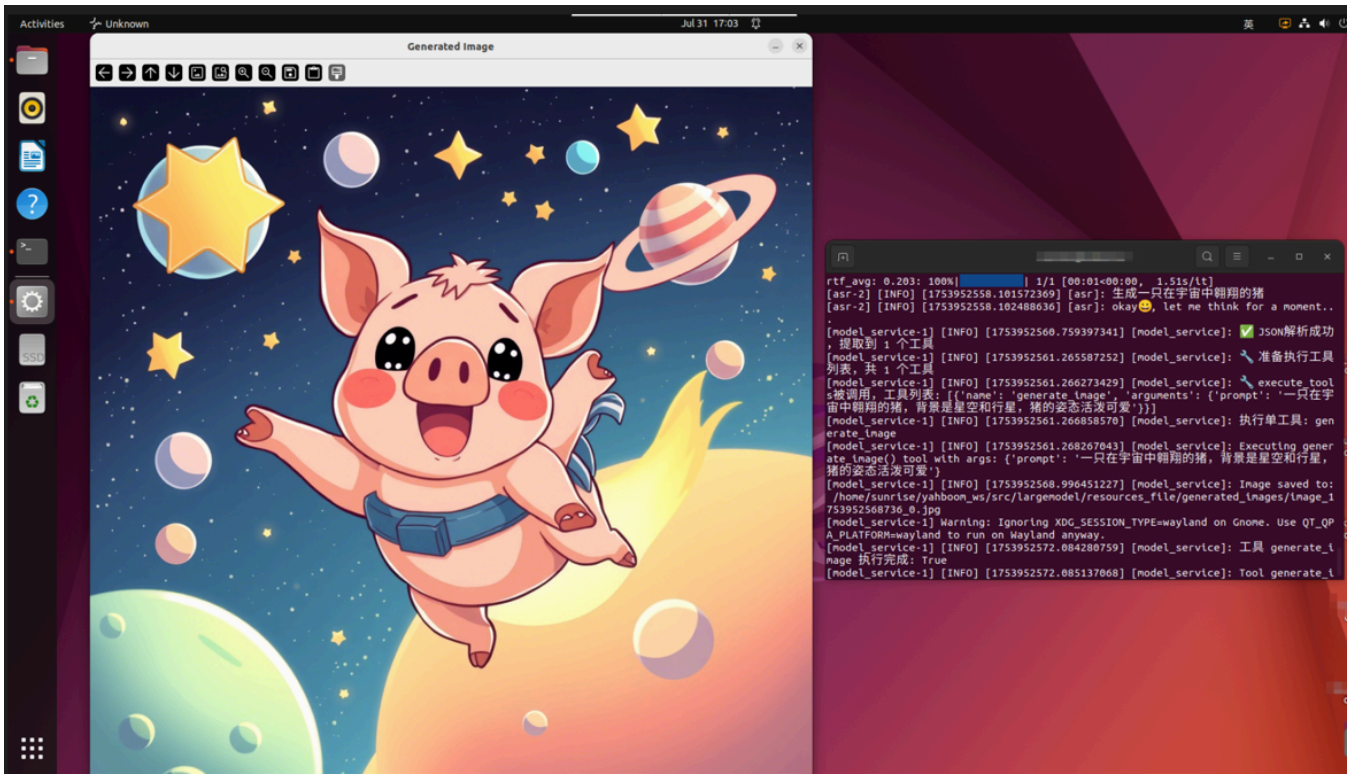
Start the text mode:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
```

In another terminal that has loaded the ROS 2 and the workspace environment, run:

```
ros2 run text_chat text_chat
```

Enter something like "generate a pig soaring through the universe". Watch the log of the main program and the image that pops up; the generated file is saved in `~/yahboom_ws/src/largemodel/resources_file/generated_images`.



3.3 The online voice input

The voice input and the announcement are controlled independently by the ASR/TTS parameters. Configure the credentials as described in the online voice ASR/TTS tutorial and set the following in `yahboom.yaml`:

```
model_service:
  ros__parameters:
    llm_platform: 'tongyi'
    use_oline_asr: True
    useolinetts: True
```

Start the voice mode:

```
ros2 launch largemodel largemodel_control.launch.py
```

After waking the device up, say "generate a picture of a pig soaring through the universe". Confirm that the terminal has recognised the voice request, that the model call succeeds, that the image pops up and that the speaker announces the result. If you only want the online LLM with an offline ASR or TTS, adjust the corresponding switch separately and configure the service as described in the relevant tutorial.

The speed of the local FastSDCPU depends on the computing power of the Jetson mainboard; the generation time of the online Tongyi Wanxiang depends on the network and on the response of the cloud service.